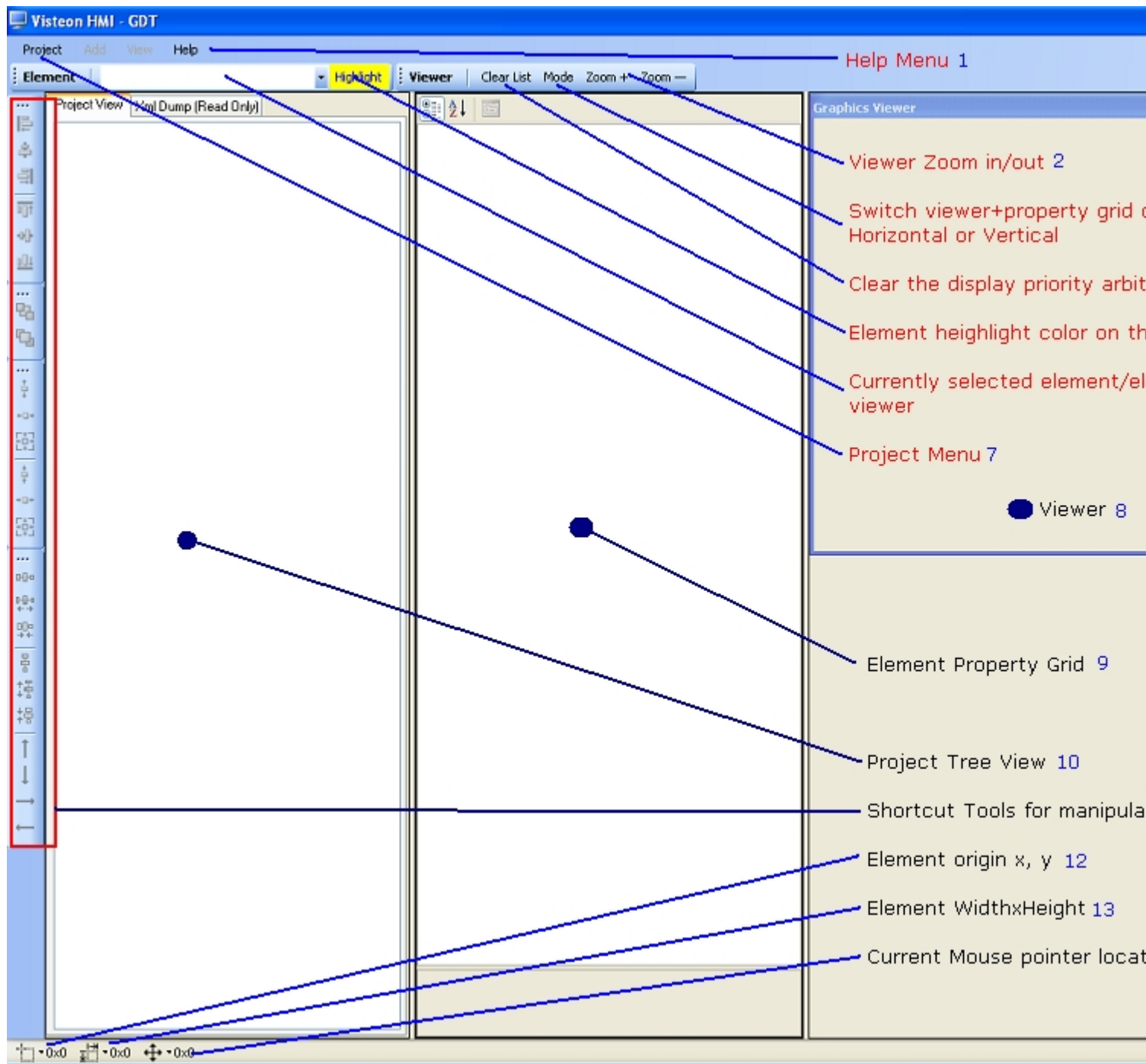


Kepler GDT User Guide

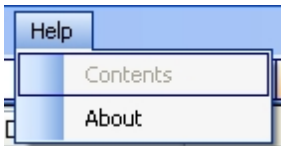
Introduction

GDT stands for Graphics Development Tool. GDT was initially developed for designing raster based graphics for the Yukawa platform and auto generate code that can be straight away integrated with the target project. Main objective was to get the best performance out of the target where there is no hardware graphics acceleration support. Following picture shows the screen shot of the GDT.



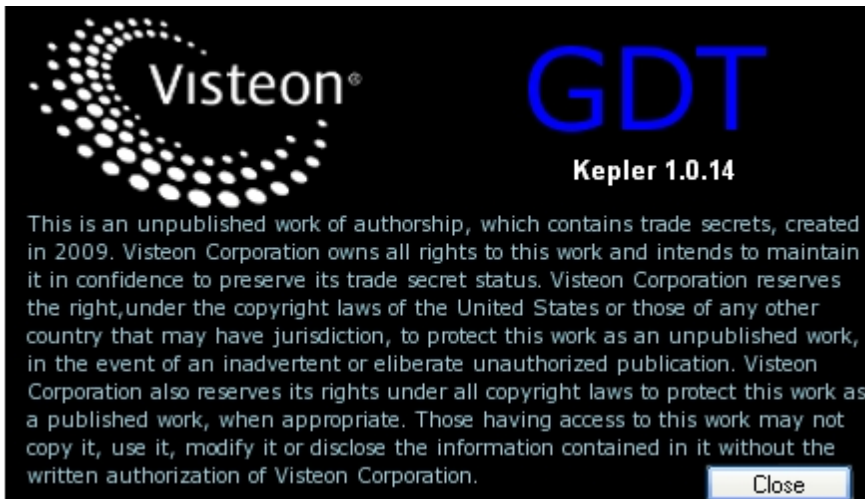
1. Help

Help menu allows the user to access this help document and view the copyright and GDT version information.



Click on Contents in Help menu to invoke the help file

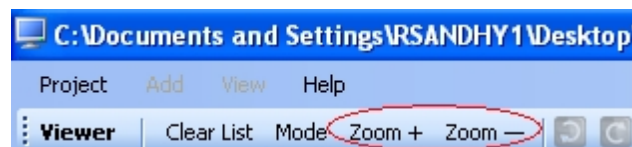
Click on About in Help menu to invoke the copyright and GDT version information. Below sample screen shows 1.0.14 version of the GDT for Kepler Platform.



[Previous](#)
[Next](#)

2. Zoom + / Zoom -

This menu allows the user to zoom into (Zoom +) the viewer to enlarge the elements so that minute adjustments in its location can be edited. Zoom can be in steps of 25% upto a maximum of 500%. Use the Zoom - to bring the viewer back to the actual size.

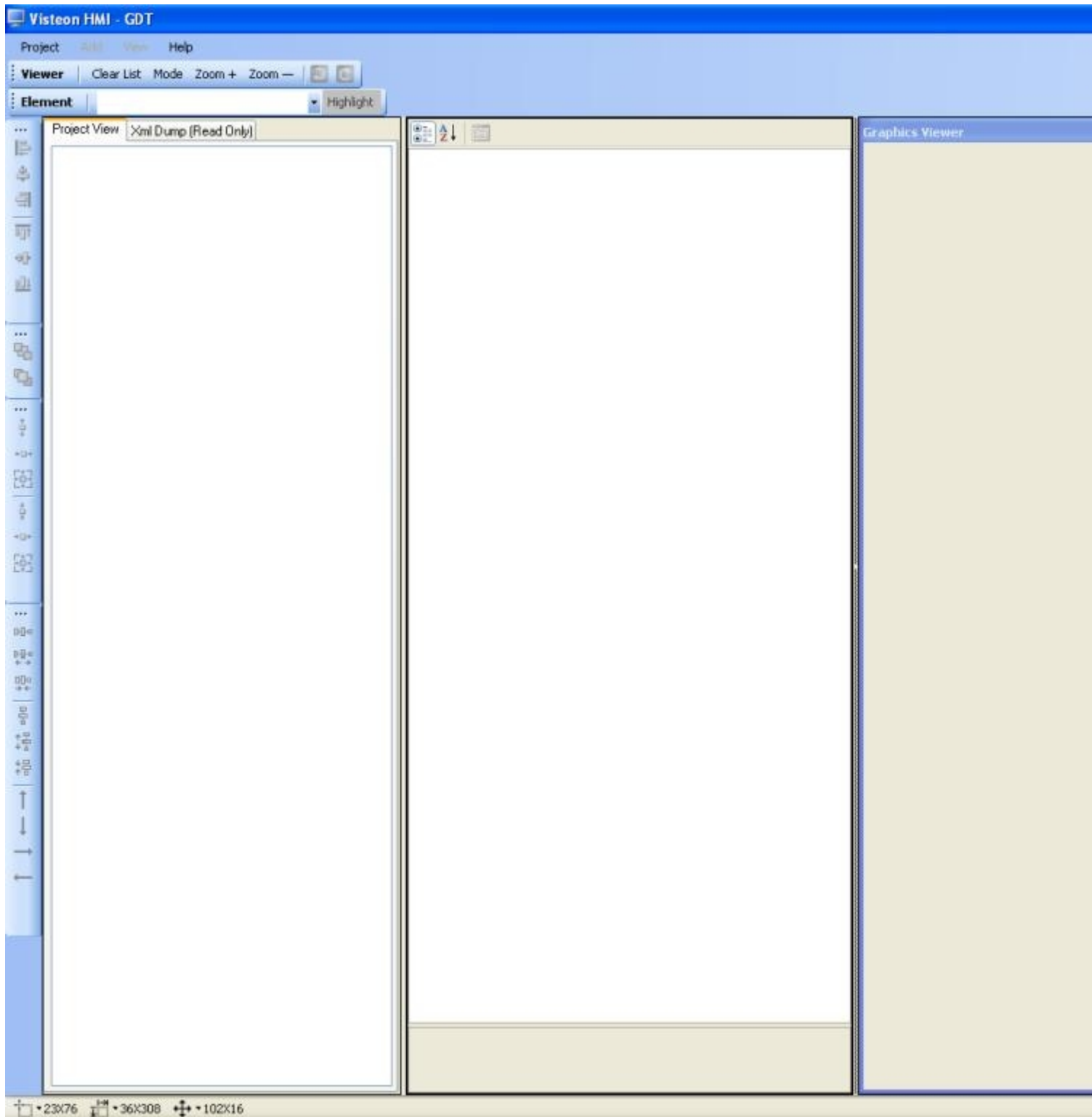


[Previous](#)
[Next](#)

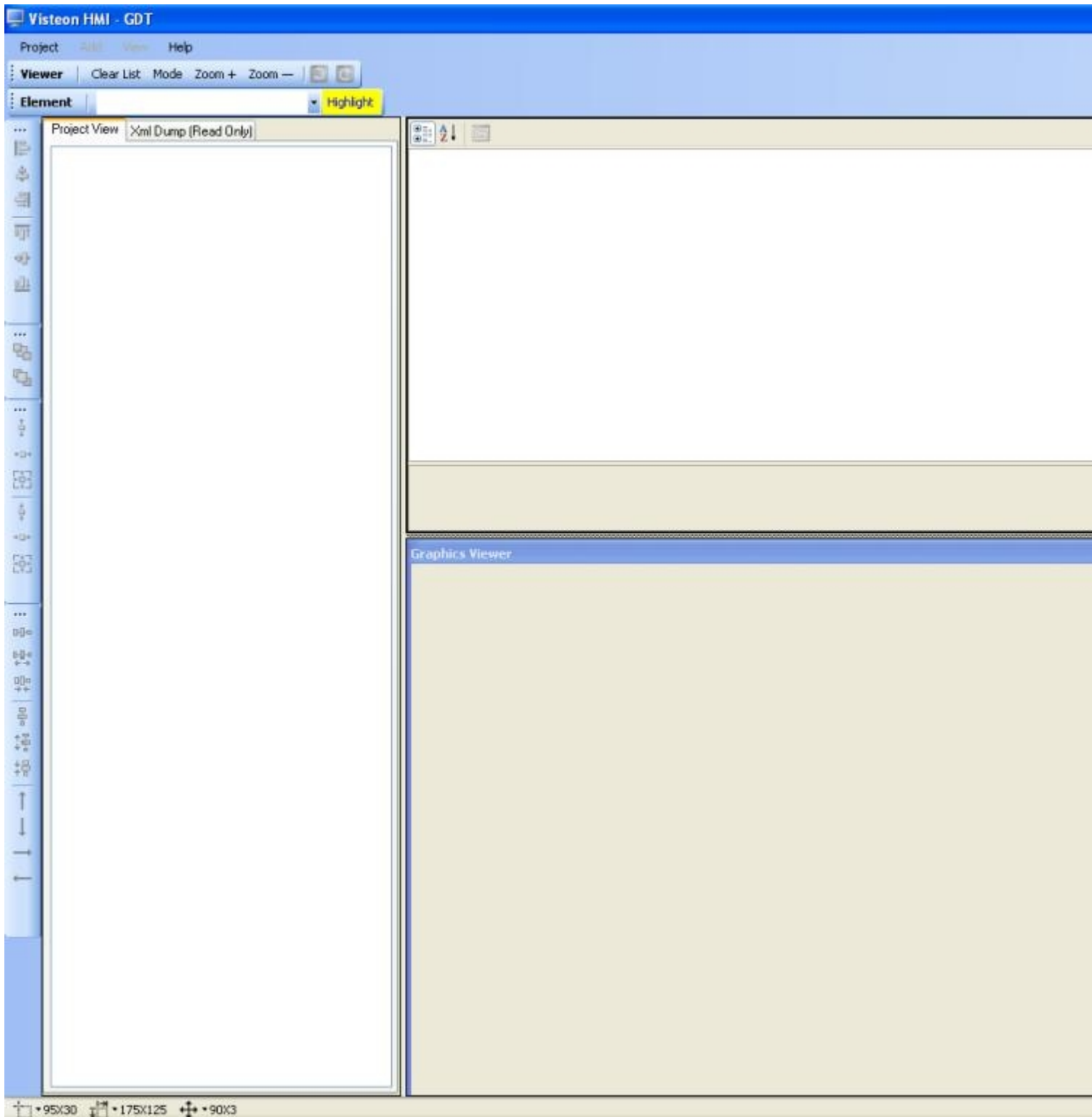
3. Mode

Mode allows the user to choose between a vertical or horizontal arrangement for the viewer and property window. Choose the one appropriate for your convenient usage.

Horizontal Mode



Vertical Mode

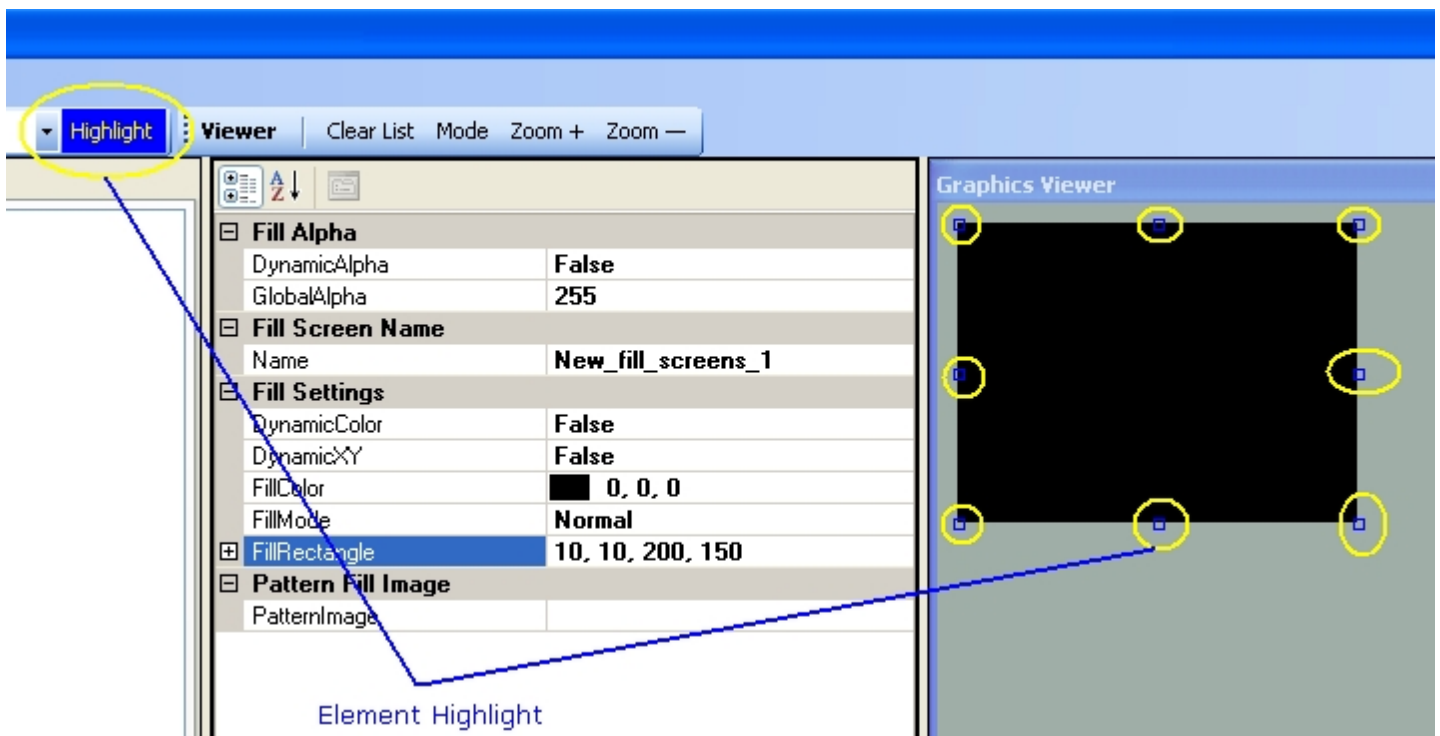


[Previous](#)

[Next](#)

4. Highlight

This allows the user to choose the highlight color for any element(eg.Image). Default is Yellow. Choose the one depending on the color shades used for your project. Clicking on this button will bring up the [color selection dialog](#) where you can choose any color of your choice.



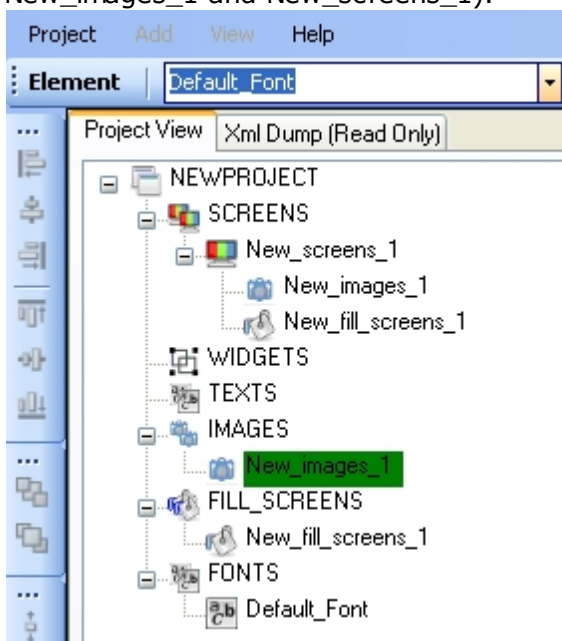
[Previous](#)

[Next](#)

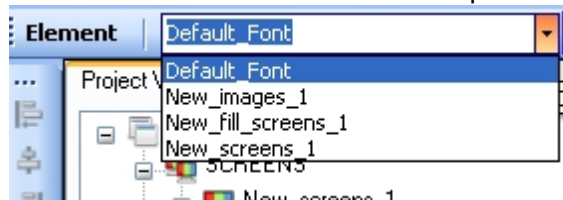
6. Element

This shows the Currently Selected Element in the viewer. As the user keeps adding new elements to the project this Element list is also updated. You can click on the drop-down list next to "Element" and choose any element from the list for viewing on the display. As soon as the user clicks on an element in the Project tree view, tool will automatically select the element. The viewer is updated to show the selected element in a [highlight](#). Only exception to this is that when you select a Screen element, the Currently Selected Element latches to it until you select another element which is not a child element of the chosen Screen. This feature allows the user to select individual child elements of a screen with it's entire element context.

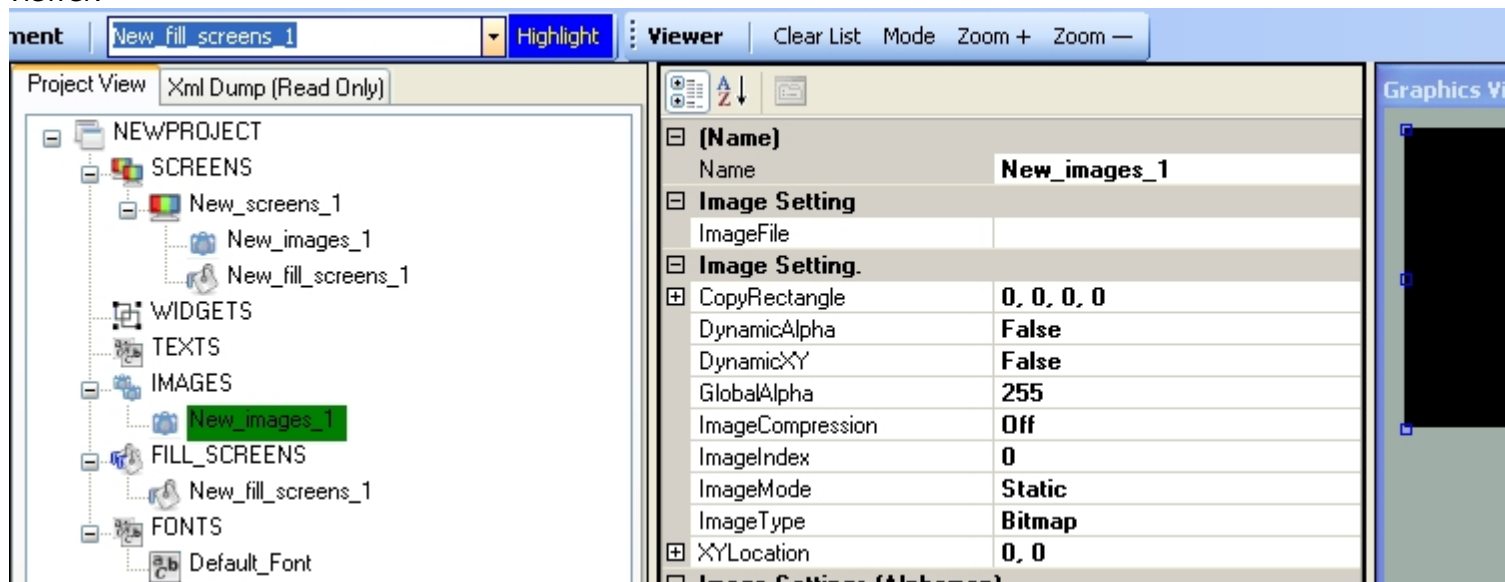
Consider the following GDT project. We have totally four elements (Default_Font, New_fill_screens_1, New_images_1 and New_screens_1).



Now if we click on the Element Drop-down and all the other elements must be listed as well.



Now choose the New_fill_screens_1 and verify if it becomes the Currently Selected Element in the viewer.



[Previous](#)

[Next](#)

6. Project

Project menu provides the following functions

1. [New](#) - Creates a new project.
2. [Open](#) - Opens the file browse dialog for choosing a GDT project xml file.
3. [Save](#) - Saves the currently opened project.
4. [Save As](#) - Opens up the save dialog where the user can choose a different file name for saving the currently open project.
5. [Close](#) - Closes the currently open project.
6. [Import](#) - Opens up a file browse dialog where the user can select another GDT project for importing in to existing project. User can graphically choose the elements to be imported in to currently open project. This will be useful for merging modules or reusing standard elements. (This feature is not yet available).
7. [Generate All](#) - Triggers code generation of currently open project.
8. [Generate Image data](#) - Triggers code generation only for images(.dat) files and associated header files.
9. [Generate GDG02 Config.](#) - Triggers code generation for config files gdg02_gfx_mgr_data.c and gdg02_gfx_mgr_data.h for the current project.
10. [Generate Font Data](#) - Triggers code generation of Fonts used in text for the current project.
11. [Project Settings](#) - Opens the project settings dialog.

Some of the menus are context based and may not be available. For example Open will not be available when a project is already open.



[Previous](#)
[Next](#)

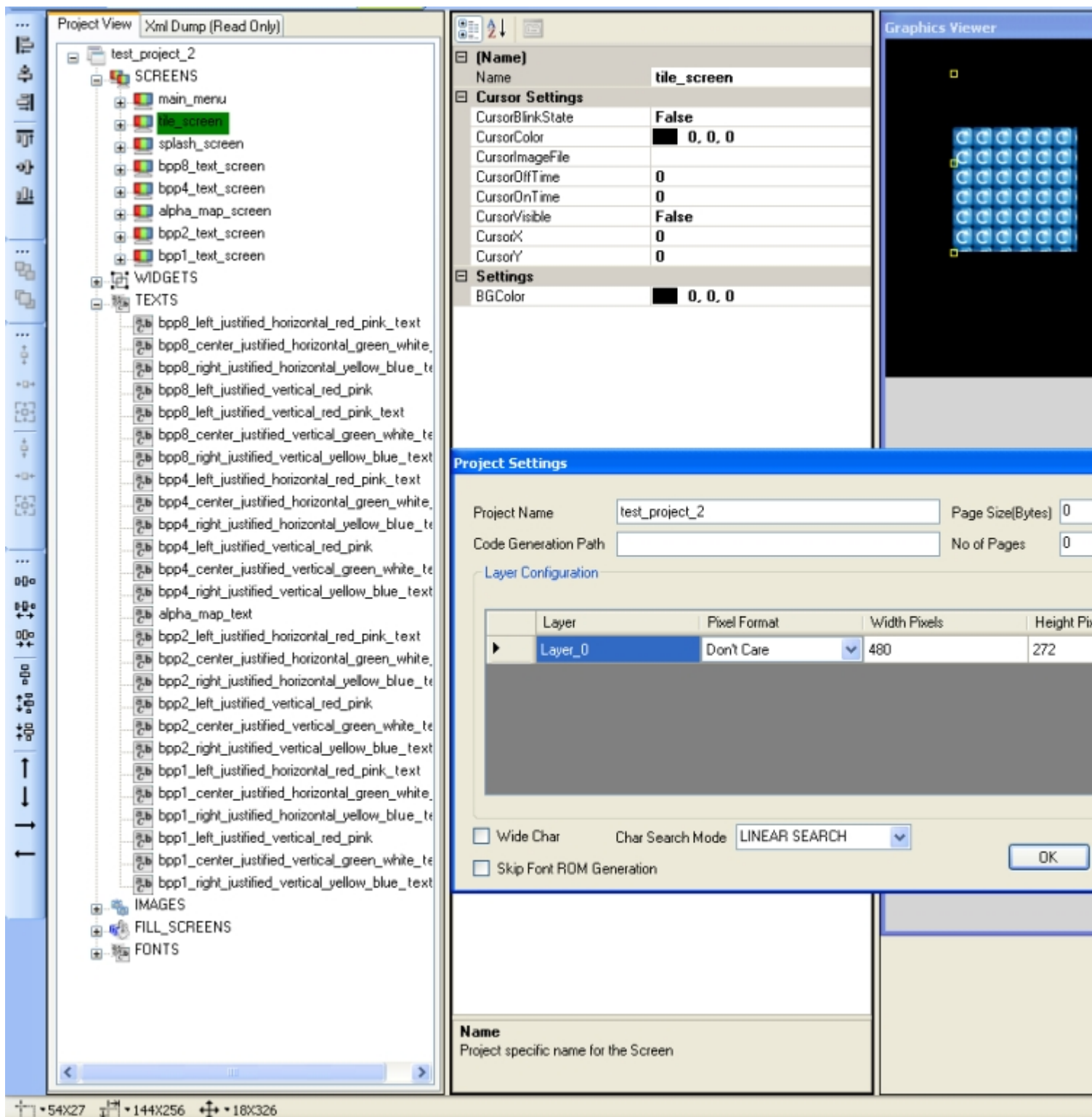
8. Viewer

Viewer provides the realtime rendering of elements based on the properties chosen such as color, XY location etc.

User can use this to verify the appearance of the elements, location of the elements etc. You can also directly click on an element in the viewer and the element would become the active element of the project. Its properties will be displayed in the property grid. Dragging of elements to new XY locations is also supported.

If the user has selected a Screen, the elements housed within the Screen will be built to provide a combined view. Viewer's initial size will be based on the display size settings entered in the Project Settings.

If you are unable to see anything on the viewer even after selecting an element, check the Project settings to see whether you have correctly entered the display Width and Height information.

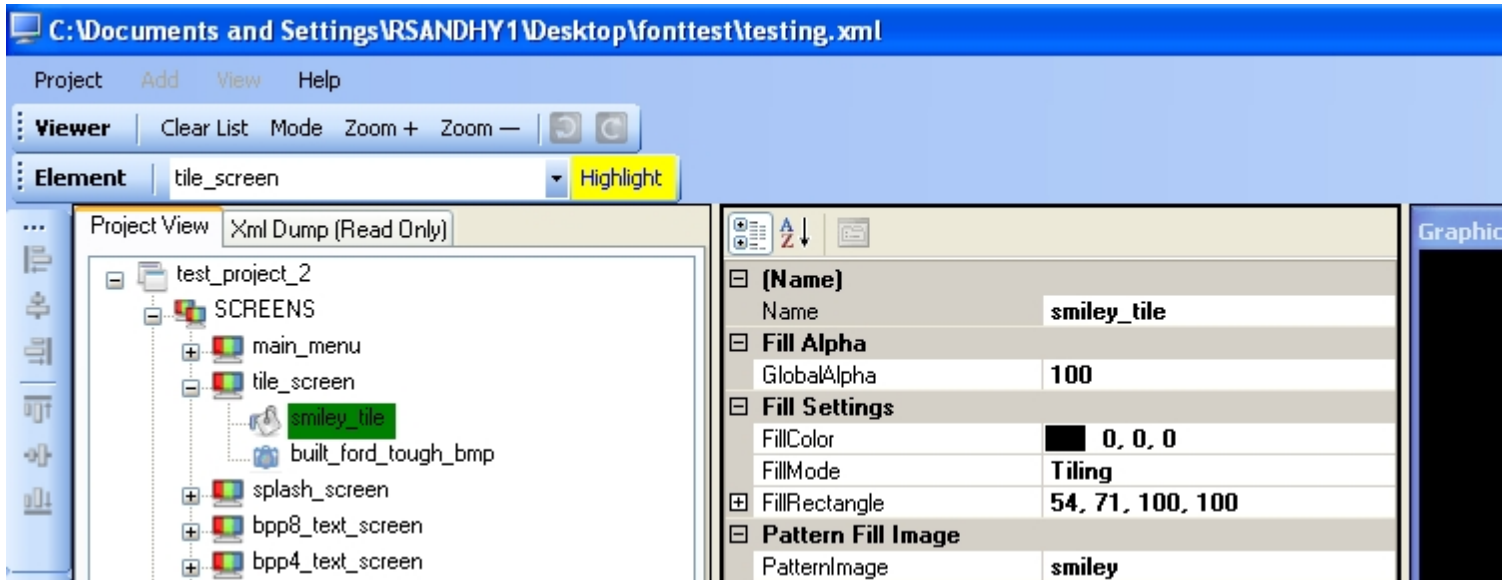


[Previous](#)
[Next](#)

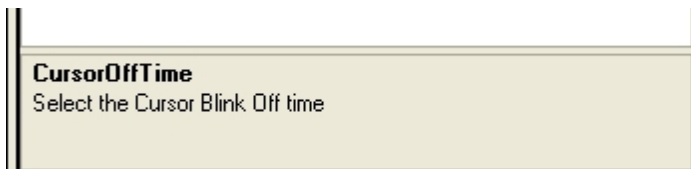
9. Property Grid

The Property grid displays the supported properties of the chosen element. User can set the appropriate settings for the chosen element. For example setting the width and height of fillscreens, choosing the image file for an image element, choosing X/Y location of the element etc.

The following image shows the property grid for a fillscreen element.



After selecting a property in the property grid you can view a quick description on it in the bottom section of the property window as below

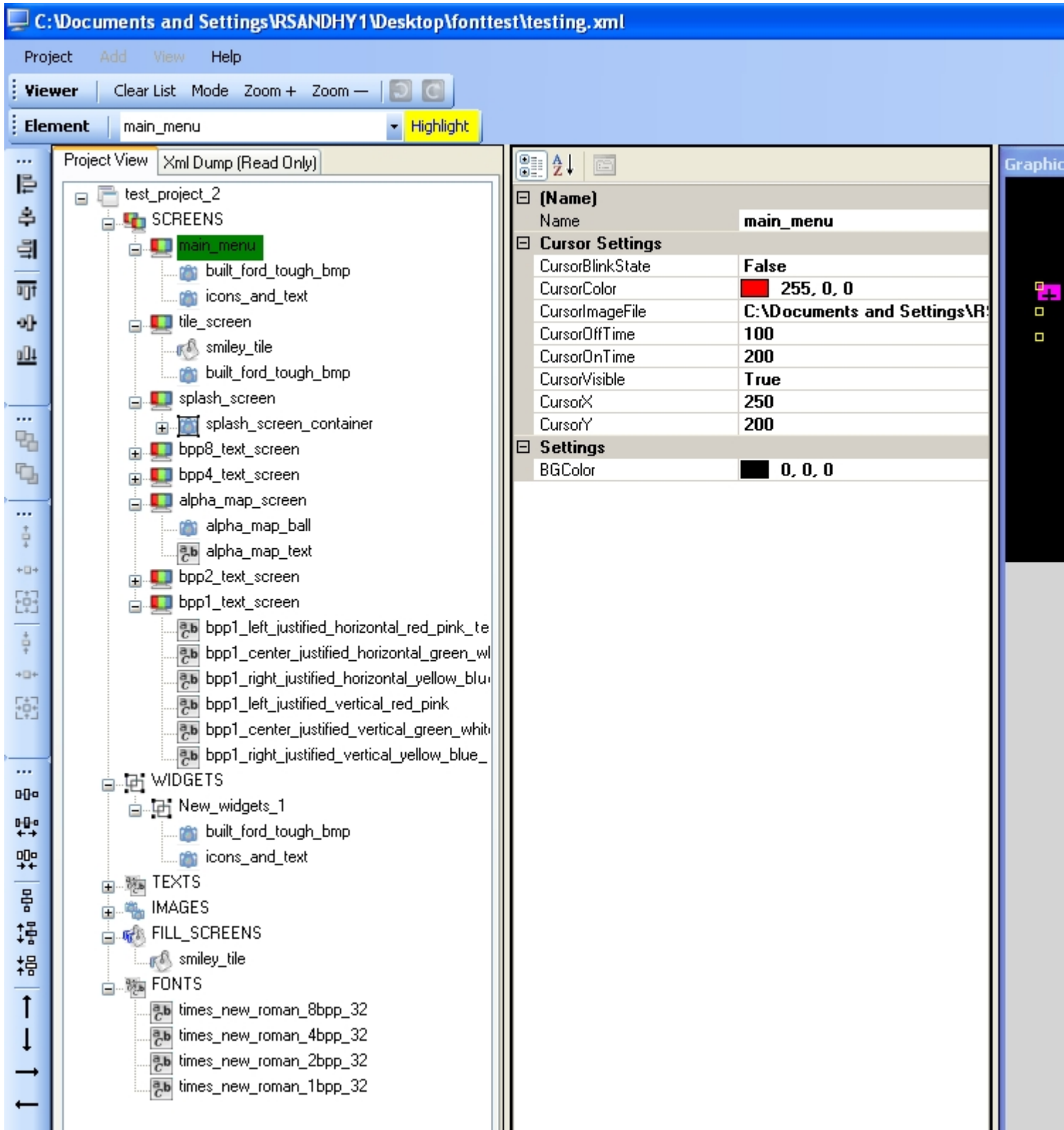


[Previous](#)
[Next](#)

10. Project View

Project View provides a hierarchical/top level view of elements in a project. The project view is presented as a Treeview where user can

1. Click to select an element.
2. Right click to open up the context menus pertaining to the selected object.
3. Rename, Copy, Delete, Drag & Drop etc.



For further details on operations possible with elements go to [Elements](#) section.

[Previous](#)

[Next](#)

11.Tool Bar

Tools bar provide some shortcut functions perform element manipulation such as

1. Alignment.
2. Building order (Bring Up/Send Back).
3. Width and Height Adjustments.
4. XY Position increment/decrement.

Tools will be automatically enabled based on applicability in a certain context.

[Previous](#)

[Next](#)

Alignment



Align Left

This tool is enabled when there are [more than one element is selected](#) in the treeview using the CTRL + Click option on the various elements.

This tool will align the Y position of the selected elements to the Y coordinate of the left most element .

Before



After



Similarly following alignments are available.

1. Align Center
2. Align Right
3. Align Top
4. Align Middle
5. Align Bottom

Try it yourself !!!

[Previous](#)

[Next](#)

Building Order



The elements contained in a Screen are built in the order in which they are placed in the container.

After including multiple elements into a screen, if the user wishes to bring an element [one level up or one level down](#), these element order buttons can be used.

[Previous](#)

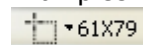
[Next](#)

11.Status Bar

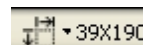
Status bar shows the following information:

1. X/Y location of currently selected element.
2. Height and Width of the currently selected element.
3. Mouse Pointer Location in the Viewer (useful in [placing elements](#)).

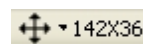
Examples



indicates that the selected element is at X=61, Y=79.



indicates that the selected image Height is 39 pixels and Width is 190 pixels.



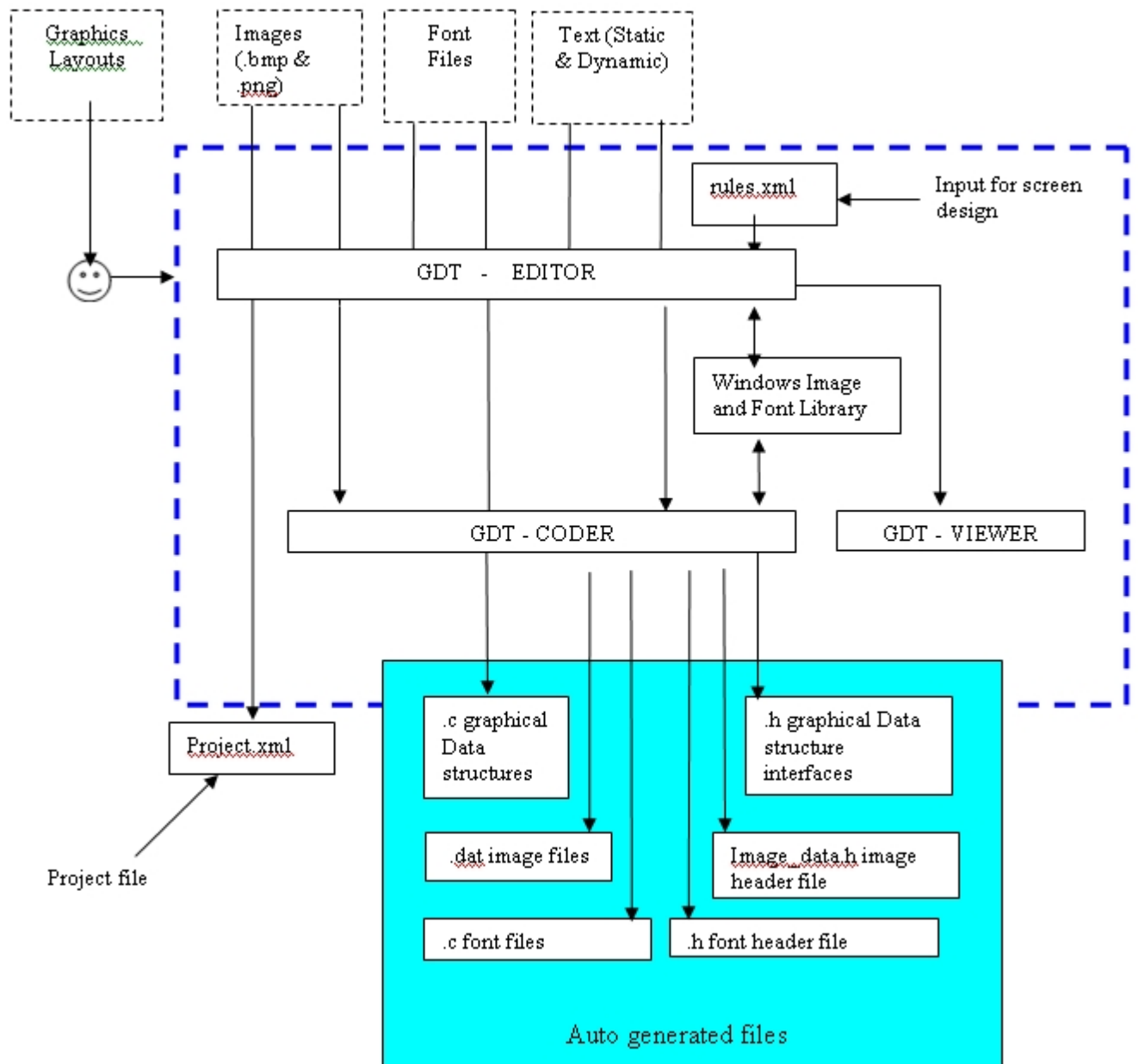
indicates that the mouse pointer is at X=142 and Y= 369 in the viewer.

[Previous](#)

[Next](#)

GDT Architecture

Following diagram shows the architecture of the GDT tool



Graphics Designer Tool (GDT) is developed using Microsoft Visual C++ .NET framework. GDT will save the project details and structure as an XML(eXtensible Markup Language).

Graphics design is done based on the principle that a [screen](#) can be constructed based on a series of drawing commands such as placing a background image, placing text at a given location etc. The designer creates a project and then adds a screen to the project. He then adds command [elements](#) to the screen, sets the element's properties such as (X, Y) location, Color etc. to get the required appearance and operational behavior for the screen. The GDT-Viewer will assist the designer in viewing the resultant screen in real time. This process is continued until all the required screens for a project are designed.

Once the screen designs are completed and the user is satisfied with its appearance and dynamic behaviour, the [coder](#) can be invoked by GDT.

The coder can be invoked in four different separate ways depending on the user's need to generate or opt out of generating certain code by clicking:

1. Generate All

2. Generate Image data
3. Generate Font data
4. Generate GDG02 Config.

The coder will parse the project's properties, rewrite them to the XML for reflecting changes and then creates following files into the working directory.

1. gdg02_gfx_mgr_data.c
2. gdg02_gfx_mgr_data.h
3. gdg02_clut_table.c
4. gdg02_clut_table.h
5. gdg02_image_data.h
6. <Image1_name>.dat
7. <Image2_name>.dat...so on and so forth until all images used in the screens are generated
8. gdg02_code_gen_warnings.txt
9. gdg02_gfx_mgr_font.h
10. <Font1_name>.c
11. <Font2_name>.c...so on and so forth until all images used in the screens are generated.
12. <NameOfXML>.xml

Please note that the GDT coder generates the pixel data for Images and the respective color lookup tables separately also(by Clicking Generate Image data). For more information, please refer
\sw_architecture\driver_information\platforms\kepler\05_product\design\kepler_sw_arch.doc

[Previous](#)

[Next](#)

Creating a Project

To start a new project in GDT first start the GDT application. Now click Project->New

This opens up the Project settings window as below:

Project Settings

Project Name: test_project_2 Page Size(Bytes): 0

Code Generation Path: No of Pages: 0

Layer Configuration

Layer	Pixel Format	Width Pixels	Height Pixels
▶ Layer_0	Don't Care	480	272

☐ Wide Char Char Search Mode: LINEAR SEARCH

☐ Skip Font ROM Generation

OK Cancel

Project Name

Enter a Project Title in the "Project Name" Field.
Example "2009 Visteon Kepler Platform Cluster"

Code generation path

Specify the path where the code needs to be generated. You can enter a relative path i.e where you are going to save your project xml file or can specify a direct path. Relative path is recommended since, when you move your files around or when someone else opens the XML from a different machine, the direct path cannot be resolved by the tool since it might not exist.

This is how you need to give the relative path. Assume that you are going to save the project in

C:\my_projects\2009_Visteon_Kepler_Platform_Cluster\Design

Case 1:-

Now if you want to generate the code in a sub folder where your design resides such as

C:\my_projects\2009_Visteon_Kepler_Platform_Cluster\Design\autocode

Use the Code Generation Path as=> .\autocode

Case 2:-

Now if you want to generate the code in another

C:\my_projects\2009_Visteon_Kepler_Platform_Cluster\Code\Src

Use the Code Generation Path as=> ..\Code\Src

Layer Configuration

Now assume that you are going to use a 480 (width) by 272 (height) display. Enter 480 in Width Pixels and enter 272 in Height Pixels.

Layer and Pixel Format are Don't cares for Kepler and you can leave them untouched.

Wide Char

If you need to support Unicode char ranges (>0xFF), then select this option so that the char length will change to 2 bytes instead of 1 byte.

Char Search Mode

Not used at the moment. You can leave this untouched.

Skip Font ROM Generation

Not used at the moment. You can leave this untouched.

Bit Ordering MSB - First

Not used at the moment. You can leave it untouched..

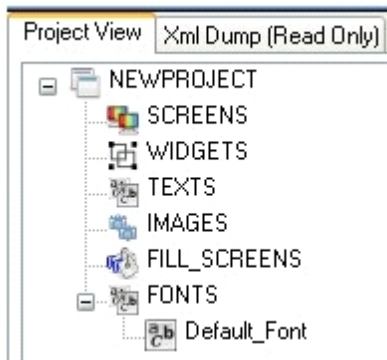
After performing above settings click OK and a new project will be created. GDT will add a default Arial 10pt font element to the project and pops a warning



Click OK and you will see the new project created in GDT. You could decide to reconfigure the default font added by GDT based on your need.

Following image shows the created project tree. You can see the default font appearing under the FONTS container. The project name specified in Project Settings dialog will become the root node name. Following the root we have SCREENS, WIDGET, TEXTS, IMAGES , FILL_SCREEN and FONT containers. User cannot delete or move these containers. Operations are possible only with the child elements created under these containers.

Note: Widgets can contain only Dynamic, static images and other widgets under them. And like other elements, they have to be used in a screen to reflect code changes.



SCREENS	- container will hold the entire user created screen elements
TEXTS	- container will hold the entire user created text elements
IMAGES	- container will hold the entire user created image elements
FILL_SCREEN	- container will hold all the user created fill screen elements
FONT	- container will hold the entire user created font elements
WIDGETS	- container will hold the entire user created widget elements

Important Note: - Properties not explained in the coming sections are not yet supported by the coder. Please leave the unspecified properties as is otherwise code generation may misbehave.

[Previous](#)

[Next](#)

Elements

GDT provides three basic elements which the user can use to create the desired graphical appearance. These are

1. Texts

a. [Font Element](#)

2. Images

3. Fill Screens

Other than this GDT provides following container elements.

1. [Screen](#)

2. [Widgets](#)

Certain elements are further classified by GDT as Static and Dynamic. Static means an element will not change after the screen is built and dynamic meaning an element can change its appearance after the screen is built.

Example a frequency text displayed will be dynamic in nature as the text value will change when ever the frequency changes. Whereas, a text named "Back" or "Exit" will stay as long as the designer desires it to be on the screen.

[Previous](#)

[Next](#)

Elements

The basic building blocks used to build the graphical appearance required are depicted below:



This could be breakdown in to following GDT elements

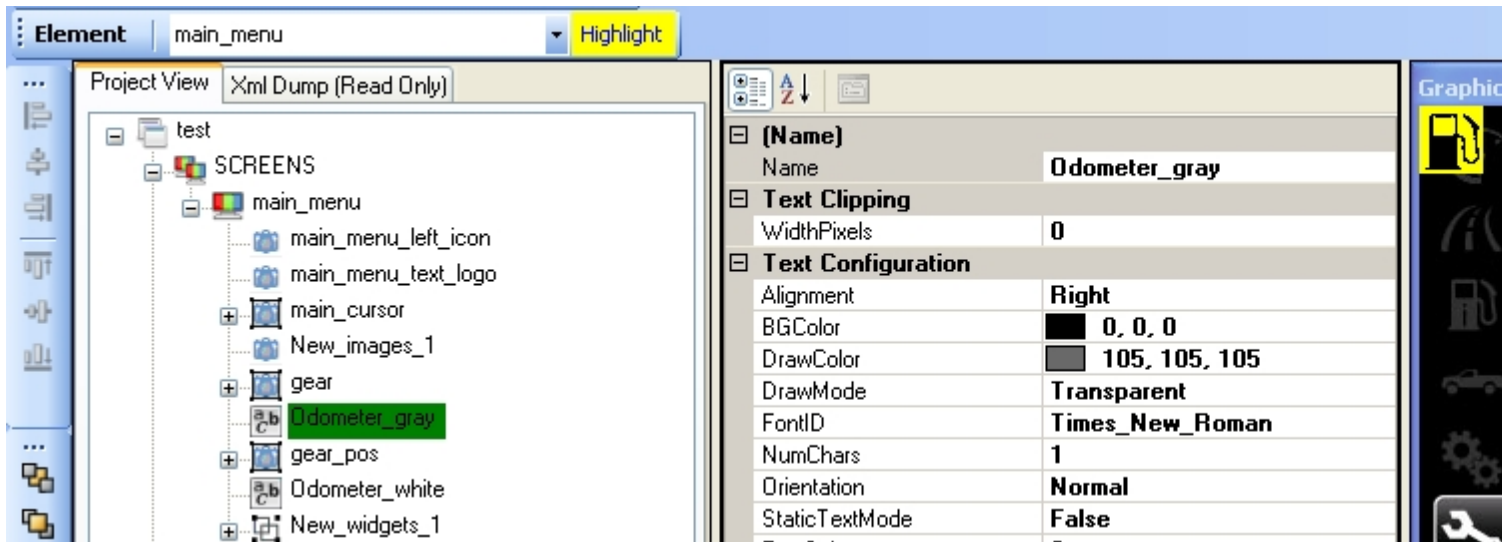
1. A black background - Fill Screen Element
2. Gauge Mode - Text Element
3. Gauge Mode Icon - Image Element
4. Trip A/B - Text Element
5. Trip A/B Icon - Image Element

And so on..

Once the items are familiar, creating these elements in GDT is a piece of cake. Arrange them under a screen, and in case you're adding more than 16 elements under a screen, you cannot. Use widgets to group similar type(eg. RGB 565) images together and drag them onto the screen.

Arrange them in the proper building order and we are done with the design.

A structured screen would look like this:



[Previous](#)

[Next](#)

Text

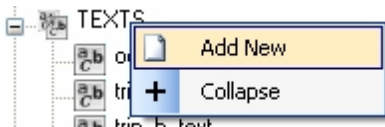
A text element can be used for displaying static or dynamic text on the display such as

1. Odometer
2. Frequency etc.

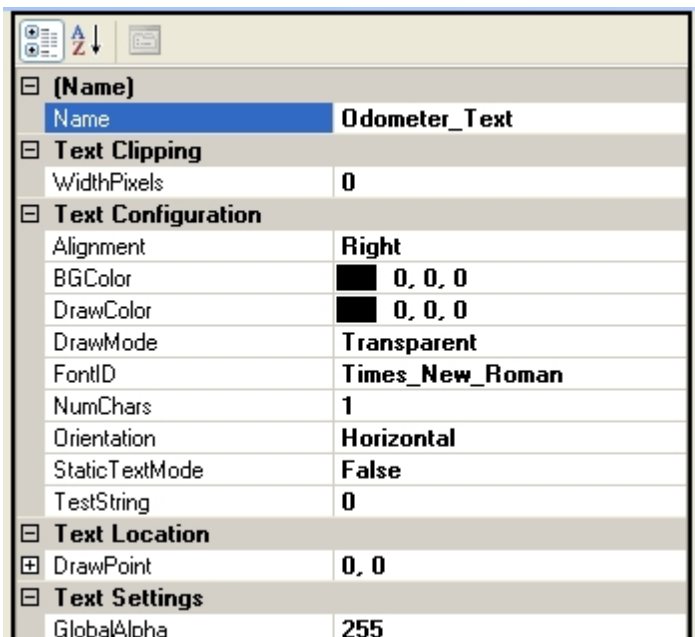
Static text :Content is fixed and you cannot change it once the code is put in to the target.
Basically the text string will be placed in ROM

Dynamic text : Contents are placed in RAM and the user can set it to any string during run time.

To add a text right click on the TEXTS container and select "Add New". This will add a new text element in to the container.



Click on the newly created text element will show up the property in the property window as below.



Name – Shows the user specified XML identifier name. You can edit by typing in a new name. This name has to be unique , cannot start with a number and avoid using special characters like "%^!@#\$(*)/\\[]{}<>.,?'"-=."

However, underscore("_") is allowed between characters.

GlobalAlpha – If this text needs to be made transparent, enter the factor here. 255 is Opaque and 0 is fully transparent.

WidthPixels – Enter the maximum width in pixels needed for this text element. Note that the text may not be displayed until width is set.

Alignment – Choose the text alignment needed "Left" or "Right" or "Center"

BGColor – Background color that will be used for the text.

DrawColor – Foreground color for this text element

DrawMode – Choose between Normal (specified BGColor and DrawColor retained), Transparent(BGColor = Screen's BGColor) and Reverse(DrawColor = BGColor, BGColor = DrawColor).

FontID – Assign the font to be used to draw this text from the font list. If the right font is not available you can add a new font in the FONTS container and set the right properties to the font. This will update the FontID list with the newly added font.

NumChars – Maximum Number of characters needed for this text. This will determine how many characters will be considered when the code is generated.

Orientation – Chose whether the text needs to be displayed Horizontal or Vertical.

StaticTextMode – Select the text needs to be static or dynamic. Static text are allocated in ROM and cannot be changed during run time.

TestString – Enter a string which GDT will display in the Graphics Viewer pane.

This string will be generated in the ROM table if the StaticTextMode is selected as True.

DrawPoint – Specify the XY location for the text.

[Previous](#)

[Next](#)

Font Element

Font elements allow the user to customize and create fonts of choice for the product. Once created these can be referenced in the Text element and the Text element will be drawn in the selected font elements settings. Font element is added as a time saving feature because a Font has multiple properties such as

1. Font name
2. Default char
3. Font Ranges
4. Anti Aliasing
5. Background color
6. Font Point size
7. Height
8. Font Style

Font Name	
Name	Default_Font
Font Ranges	
DefaultChar	0
FontRanges	(Collection)
Font Settings	
AntiAliasing	1BPP
FontHeight	16
FontType	
Name	ab Arial
Size	10
Unit	Point
Bold	False
GdiCharSet	1
GdiVerticalFont	False
Italic	False
Strikeout	False
Underline	False
Rendering	
BGColor	0, 0, 0
PreRender	False

If these settings needs to be done as part of the Text element then it would take more time since because of the redundant configuration.
This is based on the fact that we use only a limited set of Font configuration per project than the number of text elements that are used.

Create a font element and refer to the font name in the "FontID" property of text elements.

Property	Description
Name	Type a meaningful name for the font element. This name will be displayed in the drop down list of the FontID property in a Text element.
Default Char	Mention the default character to be used for missing elements. Can use 0 if not used or required.
Font Ranges	Mention the ASCII ranges of characters that the font depicts
FontType	Click this to open up the font dialog where you can choose the required font from the displayed list. Note that GDT supports only True Type Fonts which is a limitation of the .NET framework.
AntiAliasing	Choose whether the Font needs to be generated with Antialiasing levels or not. 1BPP - No antialiasing. 1 bits per pixel and one byte will contain 8 horizontal pixel information. 2BPP - 4 level Antialiased. Generates a 2BPP alphamap. Need double the ROM space than that of 1BPP. 4BPP - 16 level Antialiased. Generates a 4BPP alphamap. Needs double the ROM than 2BPP. 8BPP - 256 level Antialiased. Generates a 8BPP alphamap. Needs double the ROM than 4BPP.
BGColor	Choose a background color to be used while pre-rendering. So,BGColor is applied only when "Prerender" is true.
Prerender	Disable/enable text prerendering.

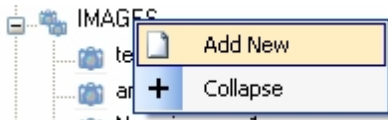
[Previous](#)

[Next](#)

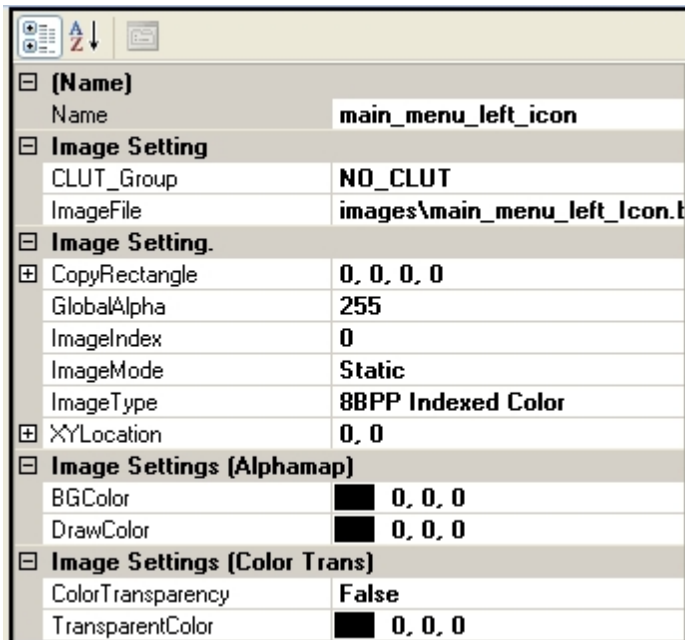
Images

An image element can be used to display images on the display.

To add an image right click on the IMAGES container and select "Add New". This will add a new image element in to the container.



Click on the newly created image element will show up the property in the property window as below.



Name – Shows the user specified XML identifier name. You can edit by typing in a new name. This name has to be unique , cannot start with a number and avoid using special characters like "%^!@#\$(*)/[]{}<>,.?\"'-+=".

CLUT Group – This setting is only valid for indexed color ImageType. This specifies the name of the Color Look-Up Table group. If specified, GDT will generate a single CLUT for all the images with same same CLUT group name. Leave it blank if a seperate CLUT needs to be generated for this image. If no CLUT needs to be generated, specify NO_CLUT. By specifying NO_CLUT, GDT will also not generate the image data. (Note that when grouping different images, we need to make sure that all the ImageType are equal.)

ImageFile – Click on the property to open up the image file browser dialog. Choose the .png or .bmp file that you wanted to assign with this image element. Note that you must save the project before attempting to choose the image file, so that GDT can correctly calculate the relative path.

Copy Rectangle – Specify the coordinates of the part of the image that needs to be cropped for generation. If no crop is required,default values to 0,0,0,0.

GlobalAlpha – If this image needs to be made transparent, enter the factor here. 255 is Opaque and 0 is fully transparent.

ImageType – Choose the color format for the image from the drop-down menu eg.RGB565.

XYLocation – Location of the image.

BGColor – Background color for an alphamap.

DrawColor – Foreground color for an alphamap.

ColorTransparency – Set to true if a transparent image is necessary. Set to false otherwise.

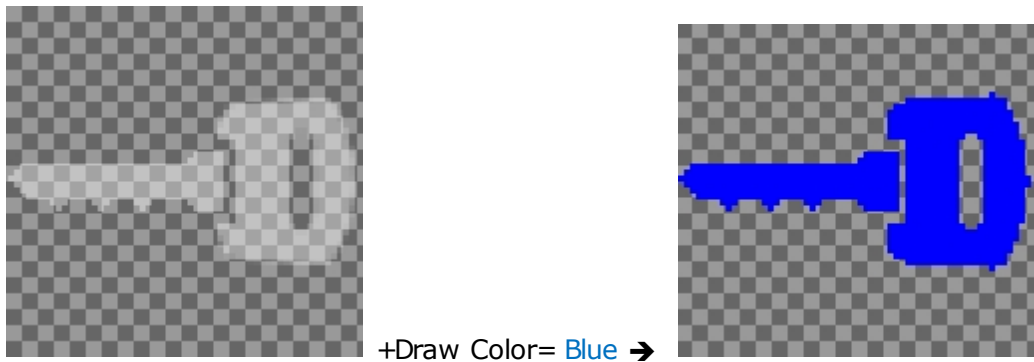
TransparentColor – Choose a transparency color. Effective only when ColorTransparency is set to True.

[Previous](#)
[Next](#)

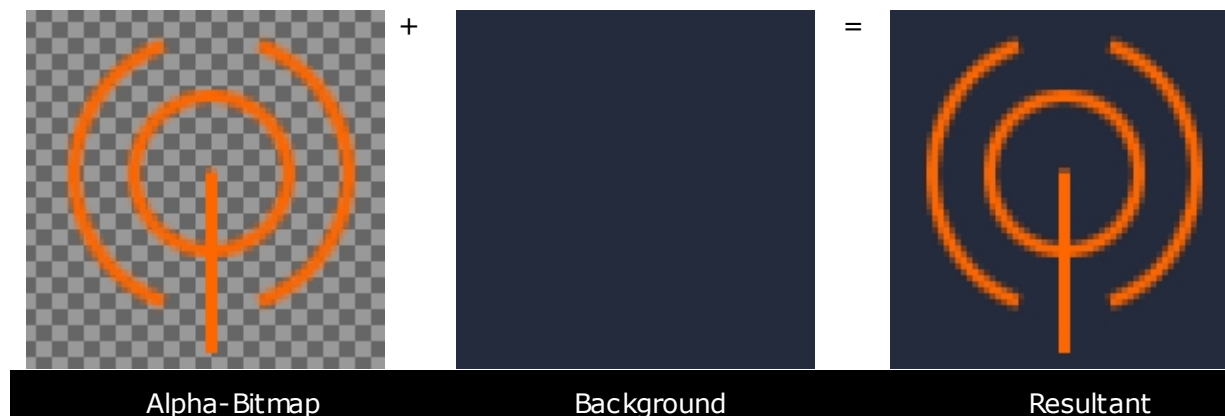
AlphaLayer

Example showing different image types.

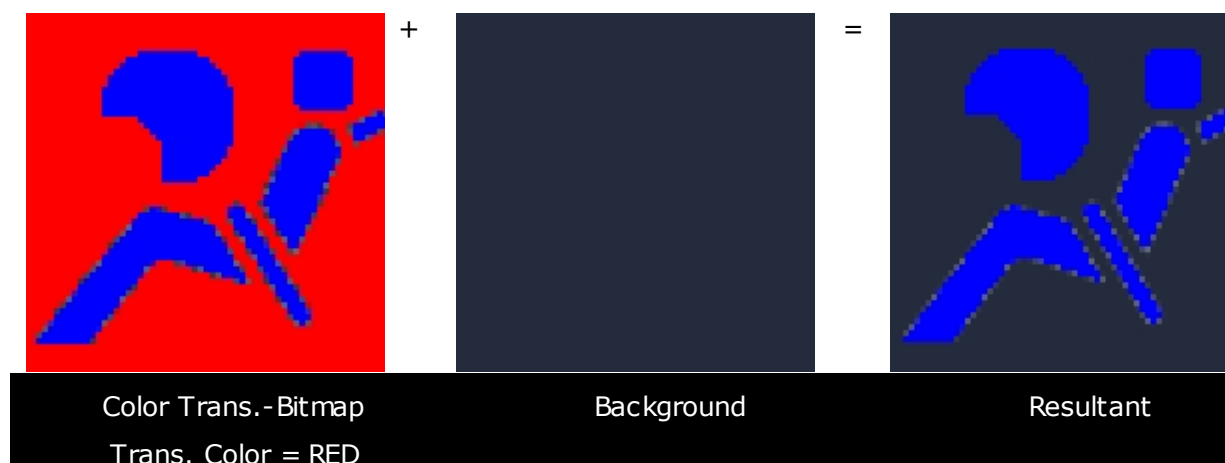
1. AlphaLayer



2. AlphaBitmap



3. Color Transparent Bitmap



[Previous](#)
[Next](#)

CopyRectangle

Example showing how the copy rectangle setting work



CopyRectangle = (1

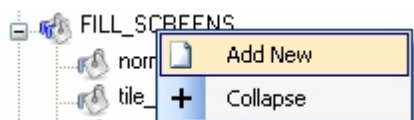
[Previous](#)

[Next](#)

Fill Screens

Fill screen element can be used to fill a specified rectangular area with a specified color or to perform a pattern fill using an image.

To add a fillscreen right click on the FILL_SCREEN container and select "Add New". This will add a new fillscreen element into the container.



Click on the newly created fillscreen element will show up the property in the property window as below.

Fill Alpha	
DynamicAlpha	False
GlobalAlpha	255
Fill Screen Name	
Name	New_fill_screens_1
Fill Settings	
DynamicColor	False
DynamicXY	False
FillColor	0, 0, 0
FillMode	Normal
FillRectangle	0, 0, 16, 1
Pattern Fill Image	
PatternImage	

Name – Displays the user specified XML identifier name. You can edit by typing in a new name. This name has to be unique , cannot start with a number and avoid using special characters like "%^!@#\$()/\[\]\{\}<>,.?\"' - +=".

GlobalAlpha – If this fillscreen needs to be made transparent, enter a value here(from 0- 255). 255 is Opaque and 0 is fully transparent.

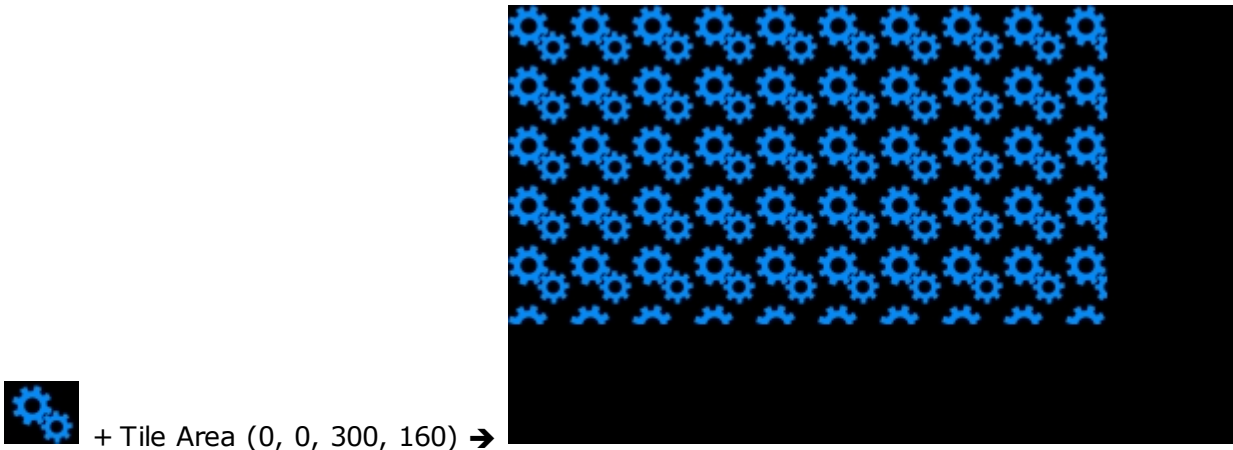
FillMode – Choose between "Normal" and "Tile" mode. In Normal mode the entire rectangle is filled with a single color chosen in the FillColor property. In tiling mode, the entire fill rectangle is filled with the specified image element in the PatternImage property.

FillColor – Specifies the fillcolor for a Normal fill mode.

FillRectangle – Specifies the fill area X, Y, Width and Height.

PatternImage – Drop-down list with a list of all image elements created so far. If the intended image is not in the list, you need to add the image in the IMAGES container and then choose it from here. This image will be tiled.

Tiling repeats an image over the specified area



[Previous](#)
[Next](#)

Container Elements

[Previous](#)
[Next](#)

Screens

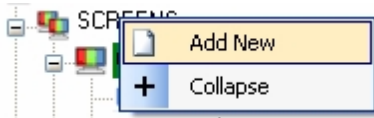
A GDT Screen means one specific view of a whole display or an area within the display. For example

Odometer View of a Driver Information display
AM Band Base screen of a head unit
A popup screen occupying a portion of the screen etc.

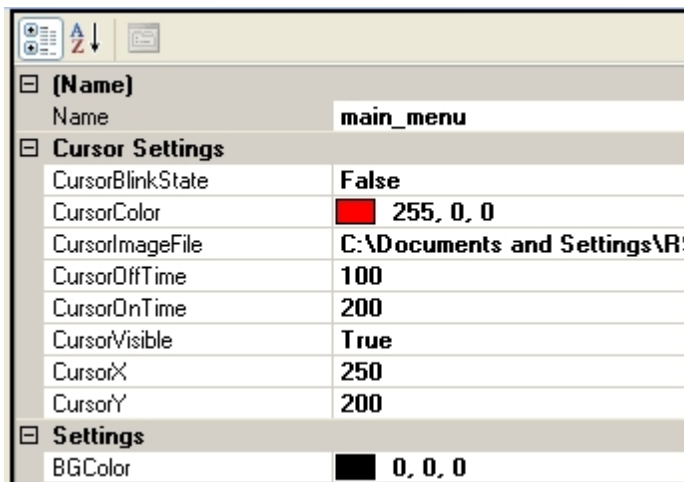
A Screen is also a group of elements. It can house only the basic elements(images,text,widgets) but no child screens.
User can create N number of screens in GDT. Each screen is a group of elements with a maximum limit of 16 elements under it.

Note that if a basic element is not used in any of the screens, then the code will not be generated for that basic element.

To add a screen right click on the SCREENS container and select "Add New". This will add a new screen element in to the container.



Click on the newly created screen element will show up the property in the property window as below.



Name – Displays the user specified XML identifier name. You can edit by typing in a new name. This name has to be unique , cannot start with a number and avoid using special characters like "%^!@#\$(*)/[]{}<>,.?\"'- +=".

CursorBlinkState – When set to true,cursor blink settings will take effect. If set to false, cursor blink is disabled.

CursorColor – Choose the color of the cursor.

CursorImageFile – Choose the image file associated with the Cursor. Leave it blank if you don't want to assign a cursor to this screen.

CursorX – Choose the cursor X location.

CursorY – Choose the cursor Y location.

CursorOnTime – Choose the On count to be used when the cursor is blinking. Base tick will be your display frame rate. For 60fps a count of 10 will give $10/60 = 166.67\text{mSec}$ on time.

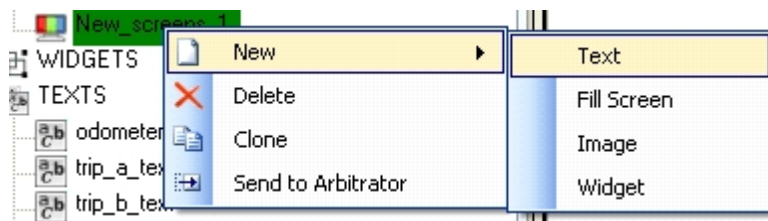
CursorOffTime – Choose the On time to be used when the cursor is blinking.

CursorVisible – Change this if you want to disable/enable cursor in the GDT viewer. No significance in code generation.

BGColor – Choose a background color for the screen through the color setting box.

You can also directly add basic elements onto a screen. To do this right click on the screen. From the context menu choose New which will open up the sub menu. Choose the element type that you wanted

to create.



[Previous](#)

[Next](#)

Widgets

A widget is short for "Window gadget". It can house similar type of elements under it. For example, Images. They can be of two types - Static or Dynamic. Static widget values (like static text) cannot be modified once the code is deployed on the target. Dynamic Widgets are yet supported by GDT.

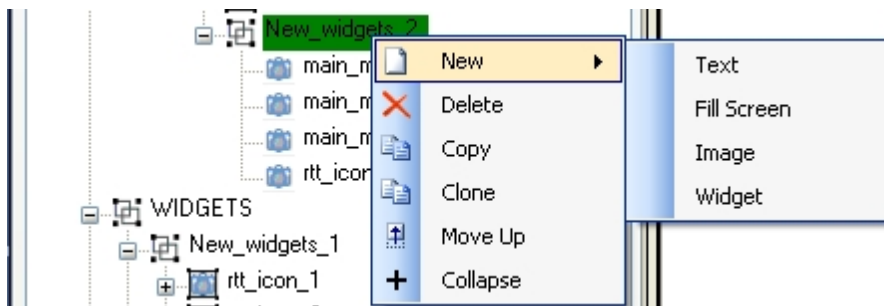
For now, a widget can contain only Dynamic and static images or other widgets under it. All the images under a widget must be of the same type (Example:1BPP indexed Color).

User can create any number of widgets for a screen. You can also create widget from another widget. Select multiple elements in the treeview;right click on the selection and select "Make Widget".

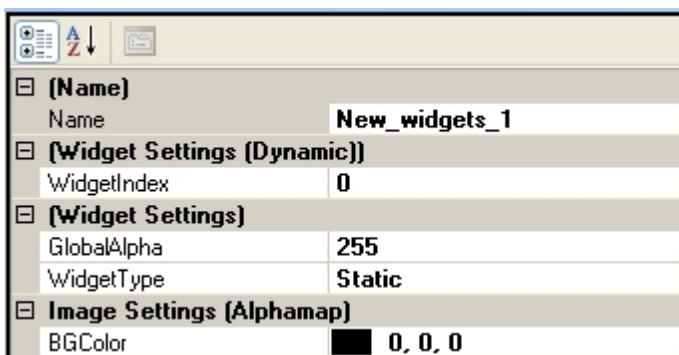
Note that if a widget is not used in any of the screens, then no code will be generated for that basic element.

To add a new widget right click on the WIDGETS container and select "New". Now choose a new element that would go into the widget.

This will add a new basic element in to a widget under the WIDGETS container.



Click on the newly created widget element.This will display the widget properties in the property window as below.



Name – Displays the user specified XML identifier name. You can edit by typing in a new name. This name has to be unique , cannot start with a number and avoid using special characters like "%^!@#\$(*)/[]{}<>.,?'"-=

WidgetIndex – This index will be reflected on the GDT viewer.

CursorColor – Choose the color of the cursor.

GlobalAlpha – Specify a value between 0 and 255 for the alpha blend.

WidgetType – Static or Dynamic. Dynamic widgets are not yet supported in GDT.

BGColor – Choose a background color for the screen through the color setting box.

[Previous](#)

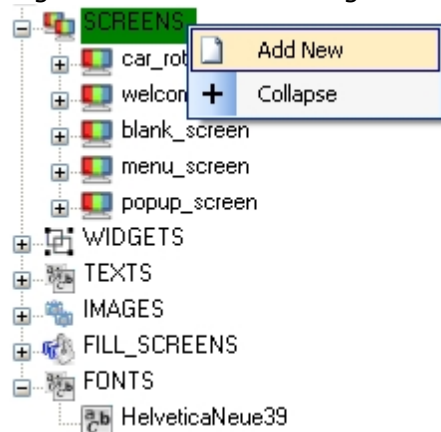
Creating Elements

User can create the elements by following methods.

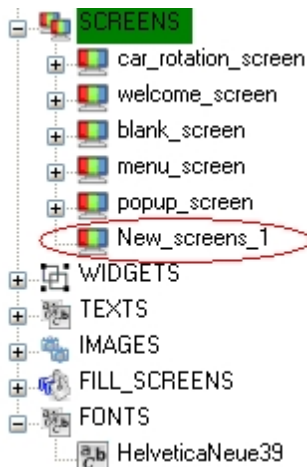
Method 1 (All Elements)

Right click on the respective element container to display a context menu. now click on the "Add New" menu item to create a new element at the end of the list. The name of the element will be automatically generated by GDT which can be [renamed](#) by the user.

E.g. to create a screen right click on the SCREENS container



Now select Add New and you will see a screen getting added.

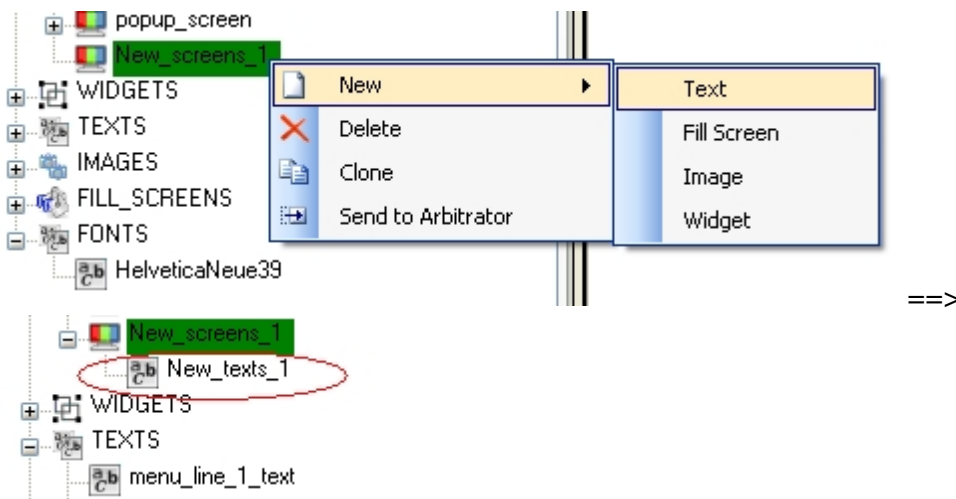


Method 2 (Only Basic Elements)

To create basic element (Texts, Images and Fill Screens) you can right click on a Screen element under which you would want to add the basic element. From the context menu chose New to display the sub menu showing Basic elements. Select the element type you wish to create. The element will be created and added to the group element.

E.g. Assume that you wanted to add a text element to the above created New_screen_1. Do the following

Right click on the New_screen_1 and select New->Text and see that New_texts_1 is created.



[Previous](#)
[Next](#)

Moving Elements

Once different elements are created you might need to move them to a screen etc. to do this

Method 1 :

Select the element you wish to move and drag it by keeping the left mouse button down. Drag it to the desired element and then release the left mouse button. When the tree view has so many items and the targeted element is outside the visible area then once you reach the top/bottom end of the tree view just keep the mouse without releasing the left mouse button for a second, the Treeview automatically starts scrolling Up/Down. Now when you see the targeted element move the mouse away from the top/bottom end to stop the autoscroll and then drop the dragged element.

Method 2:

Select the element you wish to move by clicking on it.
 Right click to bring up the context menu and select the "Copy".
 Now select the targeted element and right click. Select "Paste" from the context menu.
 You will see that the copied element is placed in the target element.

[Previous](#)
[Next](#)

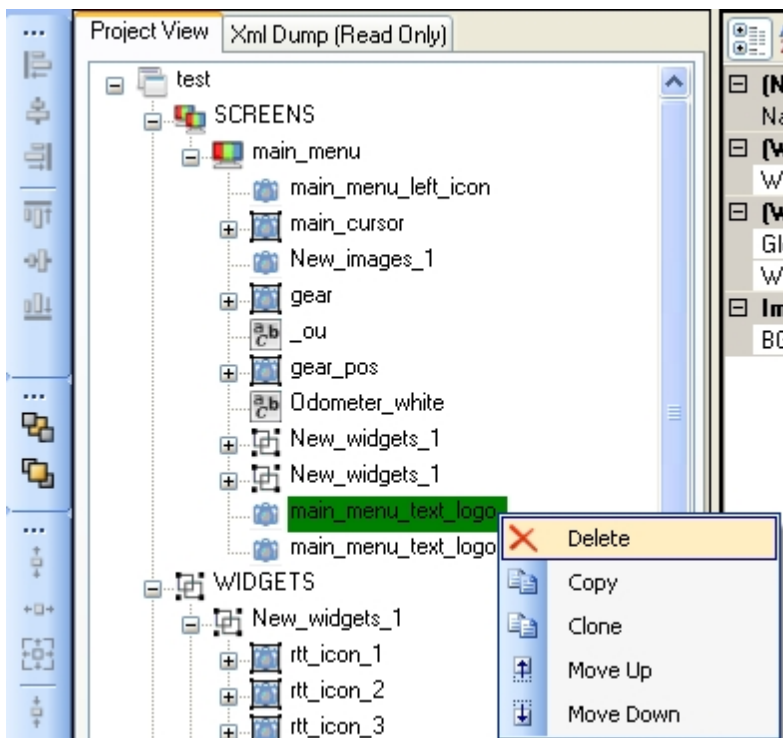
Renaming Elements

Click on the element to rename. Double click and edit the name in Treeview itself Or edit the Name property which will be shown on the property grid once you click the element.

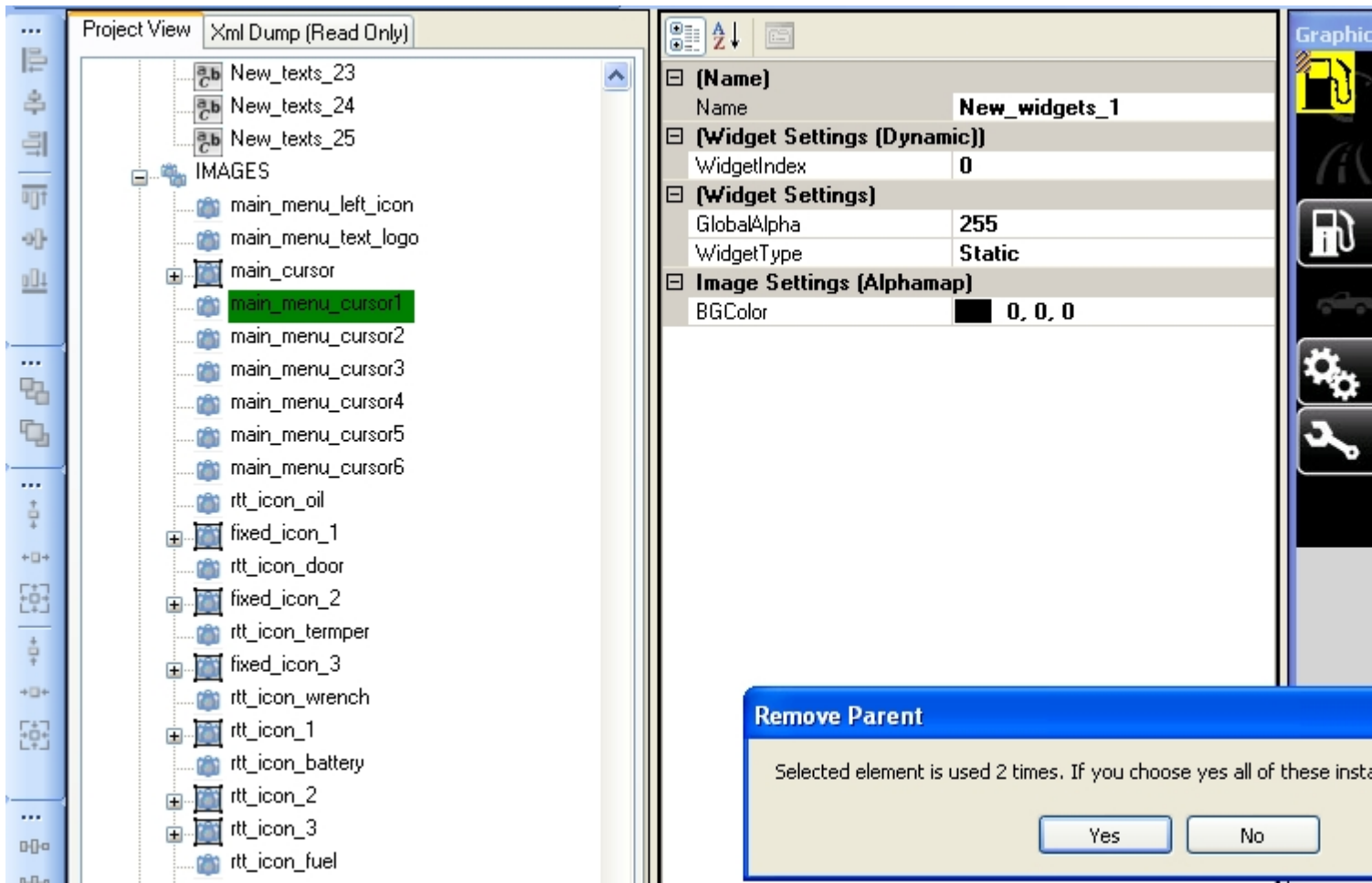
[Previous](#)
[Next](#)

Deleting Element

Right click on any element (say, Odo_image in screen1 and not under IMAGES container) and choose the "Delete" context menu to delete the element from the parent.



If delete is clicked on say Odo_image in IMAGES (it's own parent) container, all instances of Odo_Image if used in multiple screens/widgets/dynamic images will be lost forever.

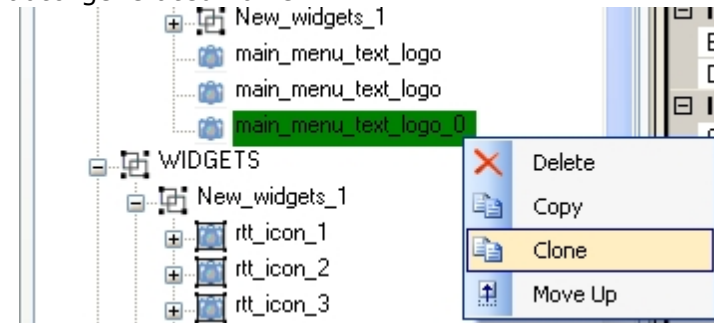


[Previous](#)
[Next](#)

Cloning Element

Right click on the element and select the "Clone" context menu.

This will create a new element with the same properties as the cloned element but with a unique auto-generated name.



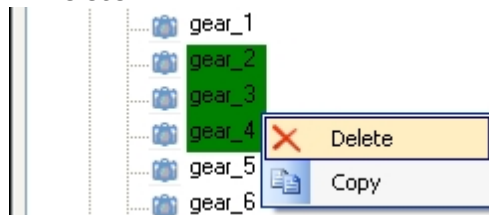
[Previous](#)

[Next](#)

Selecting Multiple Elements

Select elements with holding the "Ctrl" key to select multiple elements. You can select elements only in the same hierarchy in this method. After multiple elements are selected you could right click and choose to

1. Copy
2. Delete



In case of doing the above steps under a widget/screen, the user can choose between

1. Copy
2. Delete
3. Make Widget

[Previous](#)

[Next](#)

Arranging Elements

Consider the below example where one fill screen and a text is contained in a screen. So the viewer builds fill screen first and text next to get the below result where the text appears on top of the fill screen.

My_Screen
|_____ Blue_Fill_screen
|_____ My_Text



Now if we select the My_Text Element and then clicks the send to back tool . Now the result is that text has gone one level down.



n GDT

Also the treeview is updated to make this change as below.

```

My_Screen
|_____ My_Text
|_____ Blue_Fill_screen

```

Now if we select the My_Text Element again and then clicks the bring to front tool . Now the result is that text has gone one level up to get the original view. Treeview also gets updated to reflect this change.



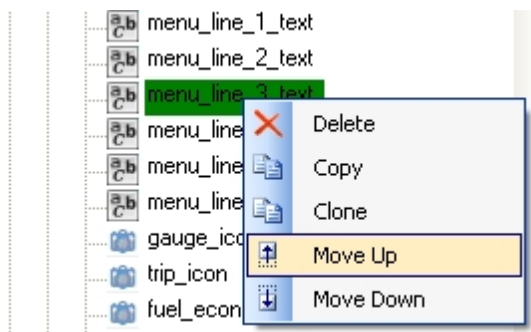
Visteon GDT

```

My_Screen
|_____ Blue_Fill_screen
|_____ My_Text

```

Now the same operation can also be done by right clicking on an element inside a Screen to get the context menu as below.



Now click "Move Up" to perform the send to back  operation.

Now click "Move Down" to perform the bring to front  operation.

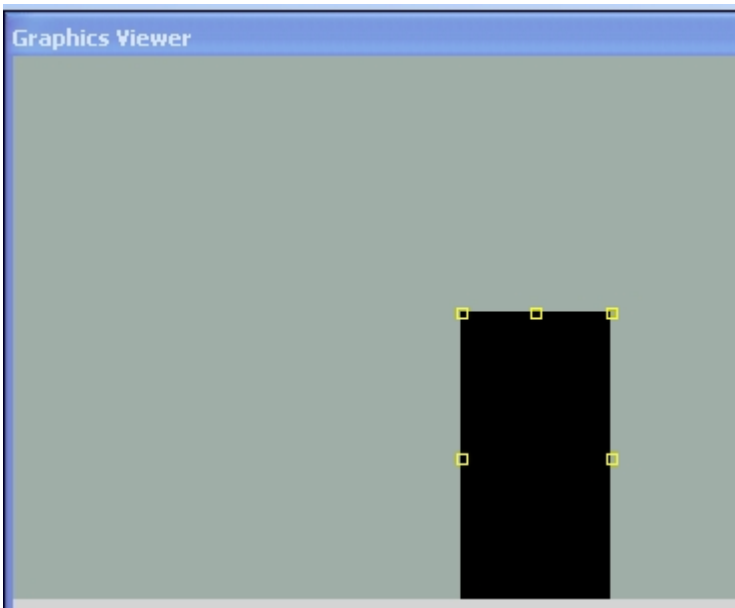
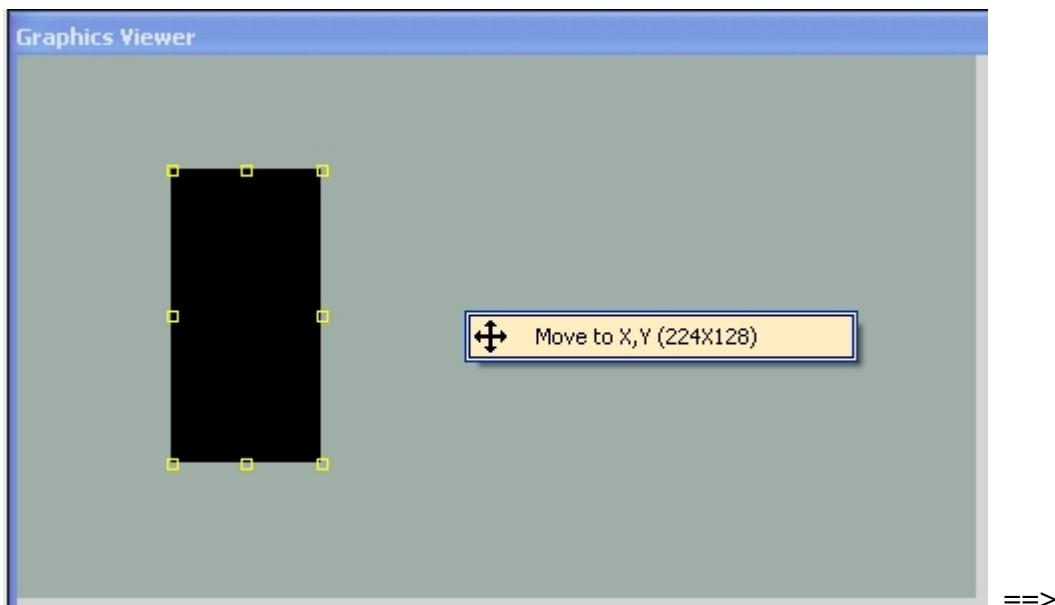
[Previous](#)

[Next](#)

Placing Element

To place an element on to a particular XY location follow any one of the methods:

Method 1 - Select the element by clicking on it. On the viewer, move the mouse pointer to the desired location by looking at the XY value at the status bar. Once you are at the desired location, right click to open up the context menu "Move to X,Y (--X--)". Select the context menu item and you will see the element moved to new location.



Method 2 - Click and select the element. Use the Up/Down/Left/Right key on the key board

Method 3 - Use the Tool Bar icons .

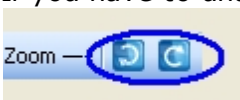
Method 3 - Click and select the element. Directly key in the XY location in the property grid.

Method 4 - Click the element on the viewer or in the treeview to select the element. Once the desired element is selected, then place the mouse pointer over the selected element and hold the left mouse button. Now drag without releasing the mouse button. Once the object is placed to the desired location release the mouse. The first select the screen element and the next click select the exact element pointed to. So always make sure that the right element is selected first and then only drag operation is performed.

Please note that if an element's XY origin becomes negative, then the entire element becomes invisible as it is beyond the pixel range specified as it is calibrated from 0 to screen resolution either ways.

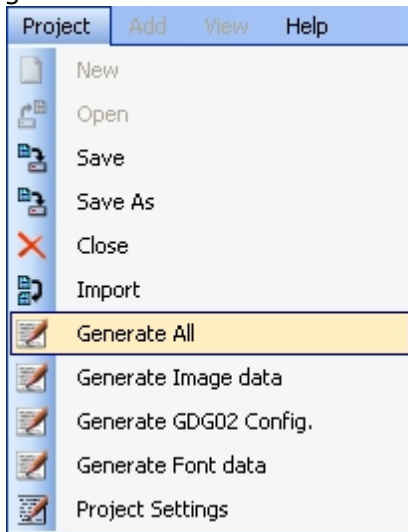
Reset the XY location to positive ranges to make the element visible again.

If you have to undo a move operation you can use the undo/redo tool.

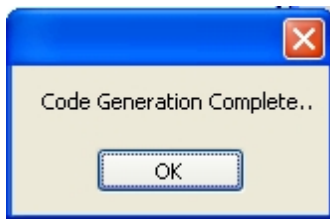


Generating Code

Once the design is completed and saved Choose the "Generate Code" from the Project menu. Code generation will start.



Wait for the popup message stating code generation complete. Could take a while if "Generate All" is clicked.



Following files will be generated on the path specified in Project Settings.

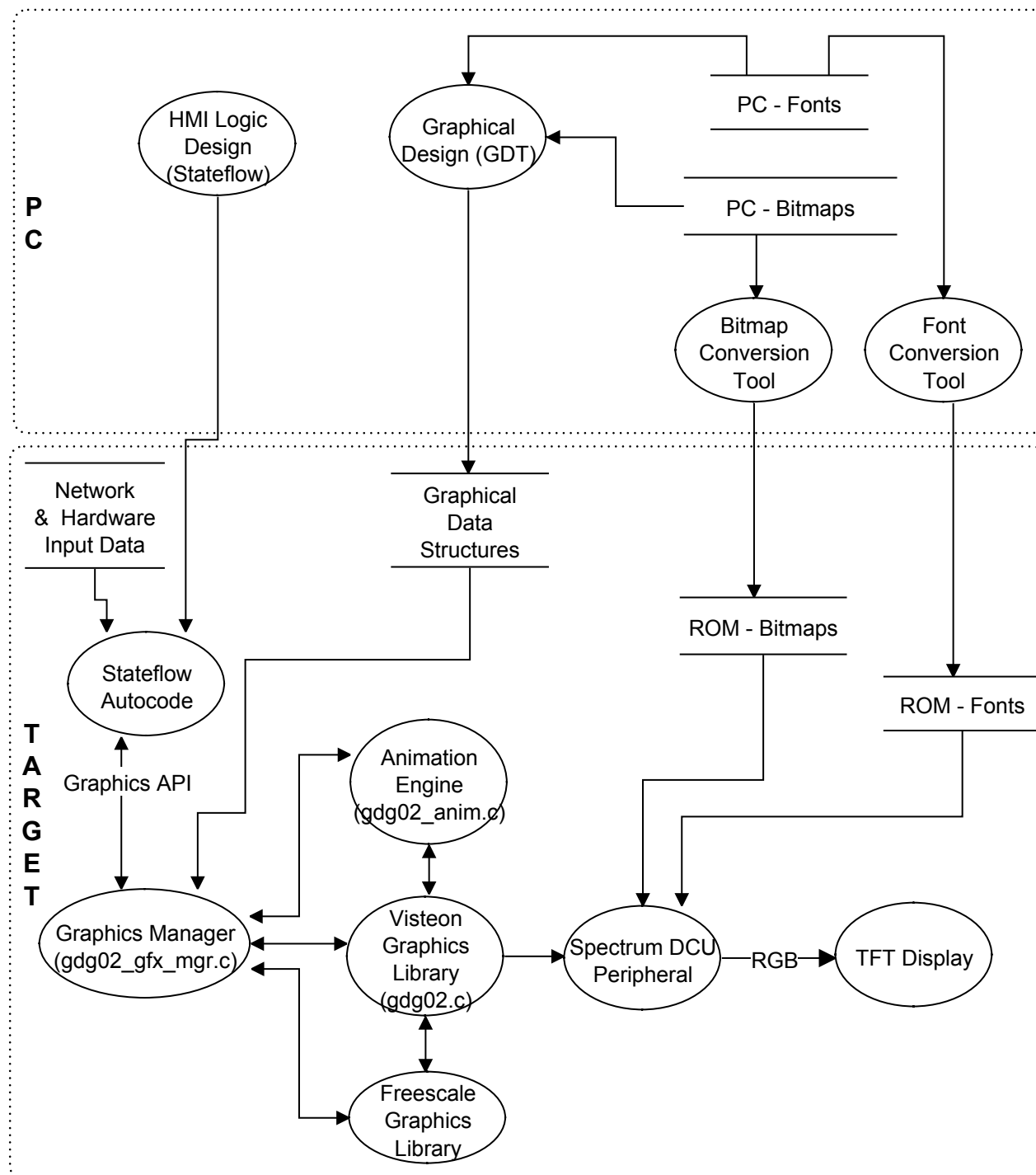
1. gdg02_gfx_mgr_data.c - Generate All and Generate GDG02 Config.
2. gdg02_gfx_mgr_data.h - Generate All and Generate GDG02 Config.
3. gdg02_clut_table.c - Generate All and Generate Image data.
4. gdg02_clut_table.h - Generate All and Generate Image data.
5. gdg02_image_data.h - Generate All and Generate Image data.
6. <Image1_name>.dat - Generate All and Generate Image data.
7. <Image2_name>.dat...so on and so forth until all images used in the screens are generated - Generate All and Generate Image data.
8. gdg02_code_gen_warnings.txt - Generate All and Generate Image data.
9. gdg02_gfx_mgr_font.h - Generate All and Generate Font data.
10. <Font1_name>.c - Generate All and Generate Font data.
11. <Font2_name>.c...so on and so forth until all images used in the screens are generated. - Generate All and Generate Font data.
12. <NameOfXML>.xml - Generate All,Generate Image data,Generate GDG02 Config. and Generate Font Data.

[Previous](#)

[Next](#)

Target Architecture

Following diagram shows the detailed view on how the GDT generated code is organized in the intended target.



[Previous](#)
[Next](#)

Application Interaction

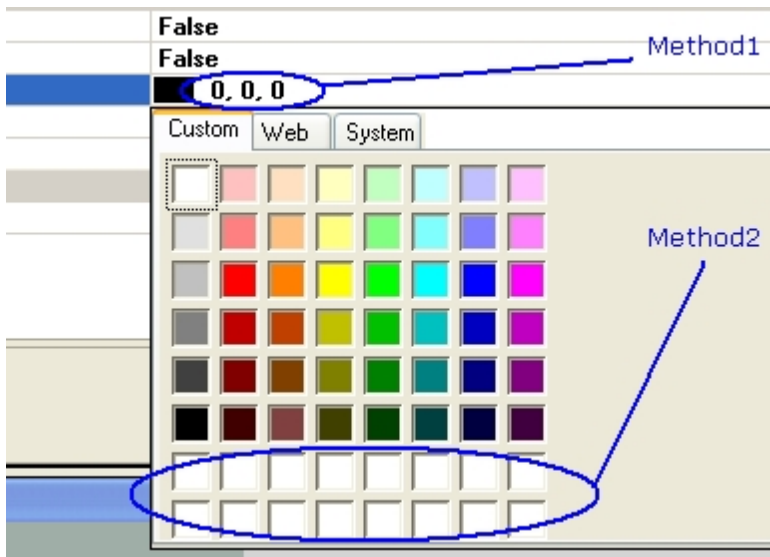
Refer to section **Graphics Manager API** in
\\sw_architecture\\driver_information\\platforms\\kepler\\05_product\\design\\kepler_sw_arch.doc

Choosing a Custom Color

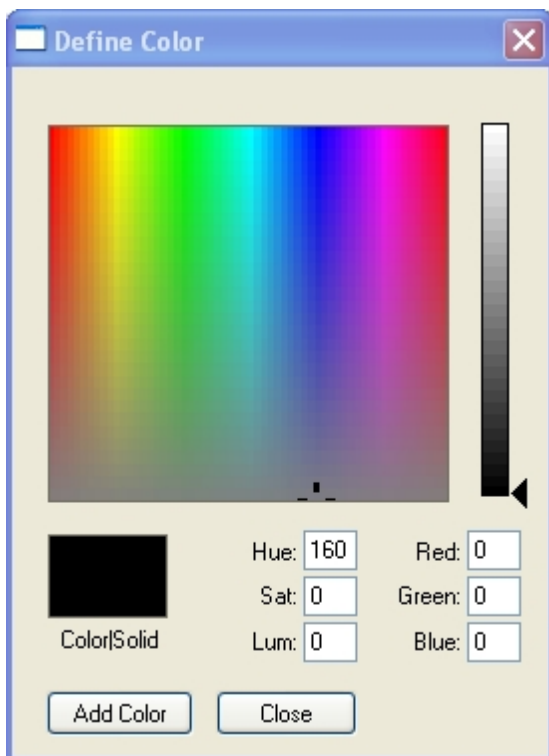
To set a custom color for an element property in the property grid use any of the following two methods.

Method 1: Enter the color in RGB888 format with a comma separating each color in decimal. For example for black color enter as 0,0,0

Method 2: Click on the drop-down menu which will show the color dialog as below.



Now right click on any of the empty white boxes to open up the define color dialog. Enter the RGB values and click Add Color button.



GDT open up a color dialog when ever the user tries to modify the color property of element highlight by clicking the [Highlight](#) button.



The color dialog will show some standard colors and when the required color is not in that standard list, follow the below to enter the required custom color. Click on the Define Custom Colors and then choose the color by directly entering RGB values.

[Previous](#)

[Next](#)